

Family Identity and Beneficiary Management Platform

Technology Stack

MVP Technical Specification

1 Technology Stack

The MVP will use a Node.js-based full-stack architecture designed for rapid development, maintainability, security, and future scalability. The system will follow a modular monolithic architecture.

1.1 Frontend

- **React.js** – Component-based frontend development.
- **TypeScript** – Type-safe frontend development.
- **Tailwind CSS** – Responsive and consistent UI design.
- **React Flow** – Interactive family tree and relationship visualization.
- **Axios** – HTTP communication with backend APIs.
- **React Router** – Client-side routing.

The frontend will provide separate interfaces for citizens and authorized government officers.

1.2 Backend

The backend will be completely implemented using Node.js and TypeScript.

- **Node.js** – Server-side JavaScript runtime.
- **Express.js** – REST API framework.
- **TypeScript** – Type-safe backend development.
- **Zod** – Request and data validation.
- **JWT** – Authentication and authorization.

- **bcrypt** – Password hashing.

The backend will follow a modular structure containing separate modules for authentication, family management, members, relationships, verification, documents, beneficiaries, schemes, and audit management.

1.3 Database

- **PostgreSQL** – Primary relational database.
- **Prisma ORM** – Database access and schema management.

PostgreSQL will store:

- Family information.
- Family members.
- Relationships.
- User accounts.
- Verification records.
- Document metadata.
- Government schemes.
- Beneficiary applications.
- Audit logs.

Prisma will be used to provide type-safe database queries and simplify database schema management.

1.4 Authentication and Authorization

The system will implement secure authentication and role-based authorization.

- JWT-based authentication.
- bcrypt password hashing.
- Role-Based Access Control (RBAC).
- Protected API routes.

- Access-token expiration.
- Refresh-token mechanism where required.

The primary roles are:

- **Citizen.**
- **Verification Officer.**
- **District Officer.**
- **Administrator.**

1.5 Family Management

The Family Management module will manage the complete family lifecycle.

It will support:

- Family registration.
- Family ID generation.
- Family Head management.
- Family member management.
- Address management.
- Family status management.
- Family lifecycle events.

Each family member will be associated with a Family ID while retaining their individual identity.

1.6 Relationship Management

The Relationship Management module will maintain relationships between family members.

Supported relationships may include:

- Father.
- Mother.
- Son.

- Daughter.
- Spouse.
- Brother.
- Sister.
- Other approved relationships.

Each relationship will have a verification status.

PENDING

|

UNDER REVIEW

|

+--+-----+

|

|

VERIFIED

REJECTED

1.7 Family Tree Visualization

The family tree will be rendered using **React Flow**.

The backend will provide family member and relationship data through REST APIs.

Example:

```
GET /api/families/:familyId/tree
```

The frontend will convert the returned relationships into nodes and edges for visualization.

1.8 Duplicate Detection

Duplicate detection will be implemented directly within the Node.js backend for the MVP.

The system will compare permitted attributes such as:

- Name.
- Date of birth.
- Parent or spouse name.
- Address.

A similarity-based matching algorithm will identify potential duplicates.

Example:

New Registration

|

v

Search Existing Records

|

v

Calculate Similarity

|

+---- Low Similarity ----> Continue Registration

|

+---- High Similarity ---> Duplicate Alert

|

v

Officer Review

The system will never automatically merge records based only on similarity.

1.9 Redis

Redis will be used for operations that require fast temporary storage.

Possible uses include:

- API rate limiting.
- Frequently accessed dashboard data.
- Temporary verification data.
- Session-related information.

Redis will not be used as the primary database.

1.10 Background Processing

BullMQ may be used with Redis for asynchronous processing.

Possible background jobs include:

- Duplicate detection.
- Document processing.
- Notification generation.
- Report generation.

BullMQ is optional for the initial MVP and can be introduced when background processing becomes necessary.

1.11 Document Management

The system will allow citizens to upload documents required for family and relationship verification.

For development:

- Local file storage may be used.

For deployment:

- Amazon S3 or another S3-compatible object storage can be used.

Only document metadata and storage references will be stored in PostgreSQL.

1.12 API Architecture

The backend will expose RESTful APIs.

The primary API structure will be:

```
/api
  /auth
  /users
  /families
  /members
  /relationships
  /documents
  /verification
  /duplicates
  /schemes
  /beneficiaries
  /audit
  /dashboard
```

Example endpoints:

```
POST  /api/auth/register
POST  /api/auth/login

POST  /api/families
GET    /api/families/:id
PUT    /api/families/:id
```

POST /api/families/:id/members
GET /api/families/:id/members

POST /api/relationships
PUT /api/relationships/:id/verify

POST /api/documents
PUT /api/documents/:id/verify

POST /api/duplicates/check
GET /api/duplicates
PUT /api/duplicates/:id/review

GET /api/schemes
POST /api/schemes/:id/apply

GET /api/families/:id/benefits

GET /api/families/:id/audit

GET /api/dashboard/statistics

1.13 Audit and Logging

The system will maintain an audit trail for important operations.

Audit events include:

- Family creation.
- Family modification.
- Member addition.
- Relationship verification.
- Document verification.
- Beneficiary application.
- Scheme status modification.
- Administrative actions.

Each audit record will contain:

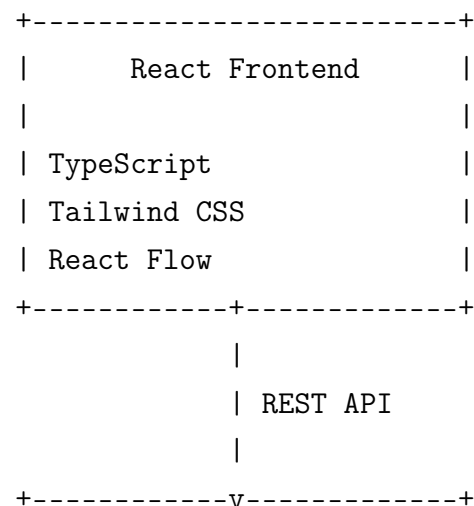
- User ID.
- Action.
- Entity type.
- Entity ID.
- Timestamp.
- Previous value where applicable.
- New value where applicable.

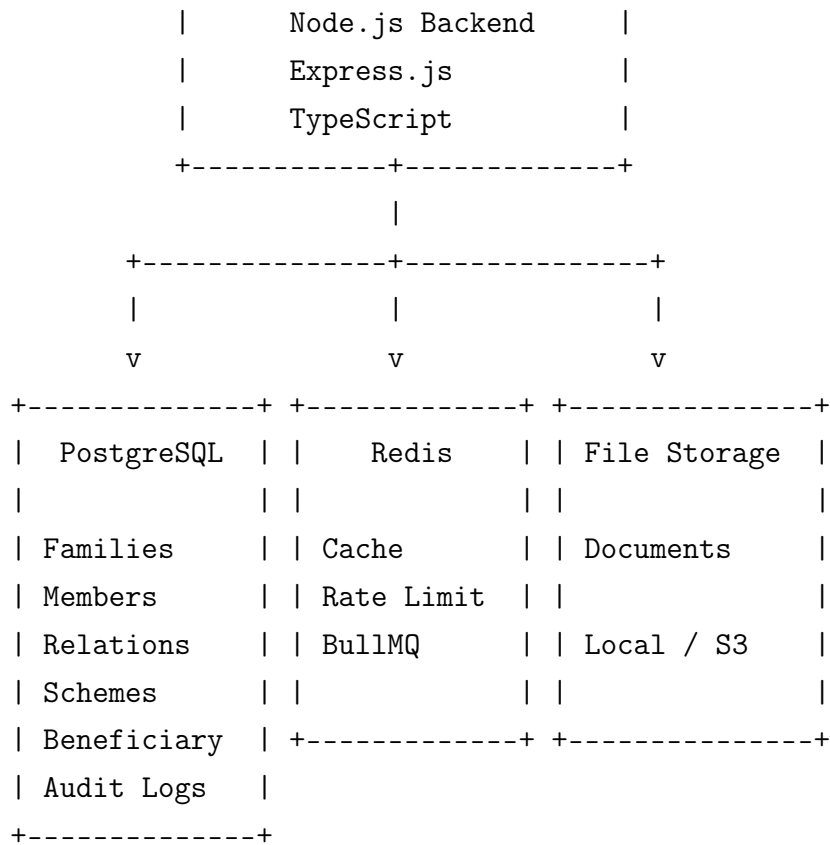
1.14 Development and Testing Tools

- **Git** – Version control.
- **GitHub** – Source code management.
- **VS Code** – Development environment.
- **Postman** – API testing.
- **Swagger / OpenAPI** – API documentation.
- **Docker** – Application containerization.

1.15 MVP Technology Architecture

The overall MVP architecture is:





1.16 MVP Stack Summary

Component	Technology
Frontend	React.js + TypeScript
UI	Tailwind CSS
Family Tree	React Flow
HTTP Client	Axios
Routing	React Router
Backend	Node.js + Express.js + TypeScript
Validation	Zod
API Architecture	REST API
Authentication	JWT
Password Hashing	bcrypt
Authorization	Role-Based Access Control
Database	PostgreSQL
ORM	Prisma
Caching	Redis
Background Jobs	BullMQ (Optional)
Duplicate Detection	Node.js + Similarity Matching

Document Storage	Local Storage / S3-compatible Storage
API Testing	Postman
API Documentation	Swagger / OpenAPI
Version Control	Git + GitHub
Containerization	Docker
Development Environment	VS Code

1.17 Architectural Approach

The MVP will follow a **Modular Monolithic Architecture** instead of a microservice architecture.

This approach minimizes development and deployment complexity while maintaining clear separation between functional modules.

The Node.js backend will contain the following modules:

1. Authentication and Authorization.
2. User Management.
3. Family Management.
4. Member Management.
5. Relationship Management.
6. Document Management.
7. Verification Management.
8. Duplicate Detection.
9. Scheme Management.
10. Beneficiary Management.
11. Audit Management.
12. Dashboard and Reporting.

The modular structure allows individual modules to be extracted into independent services in future versions if the system requires large-scale deployment.

1.18 MVP Technology Priorities

The development team should prioritize the following components:

1. Family registration and Family ID generation.
2. Family member management.
3. Relationship management.
4. Interactive family tree.
5. Relationship and document verification.
6. Duplicate family/member detection.
7. Government scheme management.
8. Beneficiary tracking.
9. Officer dashboard.
10. Audit trail.

Advanced technologies such as Kubernetes, Kafka, Elasticsearch, microservices, and complex machine learning models are intentionally excluded from the MVP to reduce implementation complexity.